

I design extensible platforms, compiler/runtime systems, and backend services where correctness depends on explicit ownership boundaries. I turn implicit dependencies and state assumptions into typed contracts, deterministic plans, and mechanically checked invariants

Single-owner architecture

UniversalToolchain: one immutable plan owns dependencies, capability selection, pass ordering, and backend routes; runtime only materializes the selected graph

State & recovery

Payments and subscriptions: explicit states, idempotency, durable inbox/outbox, reconciliation, backup/restore, and rollback

Correctness by contract

Exact manifest/ownership checks, interpreter/CIL parity, allocator verification, and differential execution

EXPERIENCE

- 2024 — present

UniversalToolchain — creator and primary developer

 - Made LanguageCompiler the single owner of composition: dependencies/ conflicts, capability selection, pass ordering, and backend routes are fixed in one immutable LanguagePlan; runtime does not make a second composition decision
 - Separated syntax → semantic binding → Bytecode → AIR → optimization → backend through explicit representation/ownership boundaries and removed the legacy reflection/runtime-profile topology
- July 1 — August 31, 2026

MCST — Compiler Engineering Intern

 - Building an LLVM pass for interprocedural linear evolution modelling of global values; unsupported CFG/call cases conservatively fall back to unknown rather than producing an unsafe inference
- 2023 — 2025

Mama Lama — Software Developer

 - Independently took event applications from requirements through deployment-ready use, defining module boundaries, component interfaces, and the complete technical implementation

PROJECTS

- UniversalToolchain / Wist2

The current exact test manifest passes with zero failures; interpreter/CIL parity, clean-consumer scenarios, and reproducible compiler/runtime contracts are verified.
- VpnMediator

A production service with controlled state transitions, backup/restore, health gates, and safe rollback; the public case study excludes secrets and user data.
- x86-64 Codegen & Register Allocation Lab

A measurable pipeline with spill/reload metrics, code size, and reproducible result comparison.

SKILLS

- Architecture

ownership/module boundaries, typed contracts, deterministic composition, extensibility, state machines, lifecycle, migrations/rollback, architecture verification
- Compilers / Systems

LLVM, CFG/SSA, dominance, program analysis, Bytecode/AIR, register allocation, x86-64 code generation
- Backend / Platform

C#/.NET, ASP.NET Core, REST/OpenAPI, PostgreSQL, SQLite, transactions, webhooks, migrations, recovery
- Verification

C++23, C17, Rust, Python, Linux, CMake, differential/ metamorphic testing, reference oracles, ASan/UBSan

EDUCATION

Software Engineering student at HSE University

RECOGNITION

First-degree diploma and the Grand Prize “Perfection as Hope” for UniversalToolchain

Moscow / remote